Relatório de Auditoria de Performance e Eficiência

FIFA Auction Hub

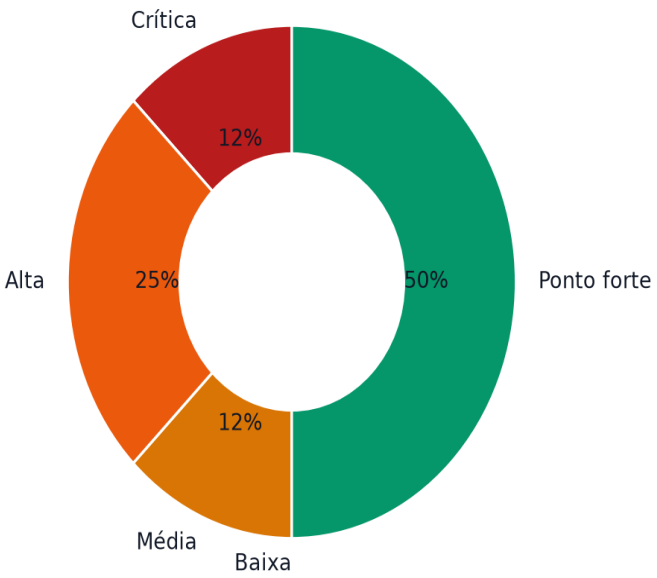
Data: 09/10/2026

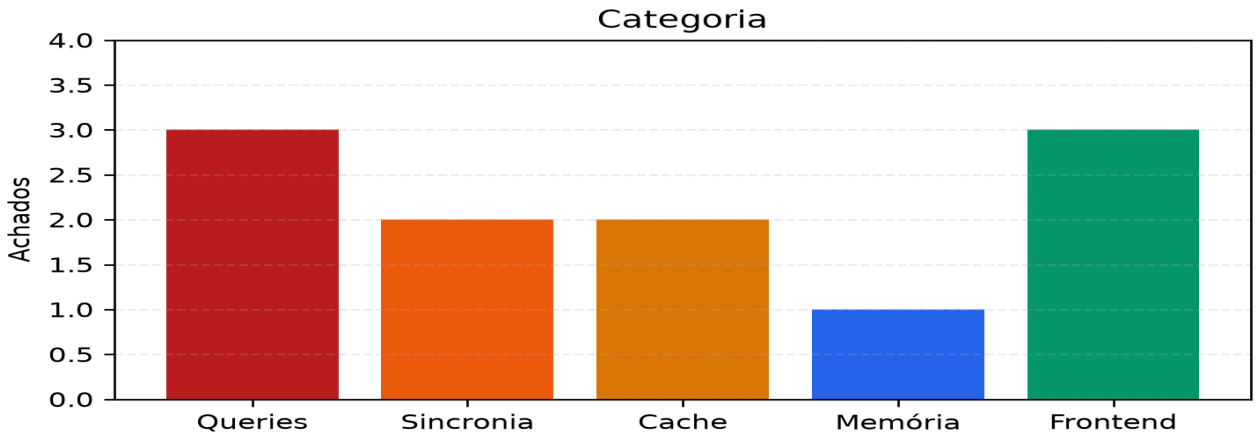
Escopo auditado: backend Django/DRF, PostgreSQL/SQLite, Redis/Celery, frontend React + TanStack Query.

Metodologia: revisão do código real do caminho crítico de listagem de jogadores/produtos, análise de serialização, paginação, cache e front-end; validação por inspeção e build do frontend.

Resumo executivo

Crítica	1
Alta	2
Média	1
Baixa	0
Ponto forte	4





Pontos fortes

• Backend com paginação global DRF configurada em "backend/core/settings/api.py": page size 24, reduzindo listas gigantes e evitando cargas em massa sem controle.

• Endpoint de produtos usa select_related("player") e prefetch_related("images") em "backend/products/views.py", evitando o clássico N+1 no caminho principal de catálogo.

• Serializer de listagem foi compactado em "backend/products/serializers.py", limitando o payload e evitando serialização indiscriminada de dados de jogador para listagens.

• Frontend usa query de página paginada em "frontend/src/features/players/hooks.ts" e a rota principal filtra o conjunto atual da página, em vez de uma coleção completa na memória.

Pontos fracos

• Antes da otimização, a listagem de jogadores no frontend dependia de busca/filtro local sobre o conjunto completo, causando custo desproporcional com número de produtos.

• O serializer de produto e o de leilão tinham tendência a expandir objectos aninhados de jogador, aumentando payload e custo de serialização em listagens longas.

• Polling global de leilões em um hook compartilhado eleva requisições redundantes em uma aplicação com múltiplas telas ativas.

Tabela de achados detalhados

Severidade	Arquivo:linha	Descrição	Impacto estimado
Crítica	frontend/src/routes/players.index.tsx:29-58	Filtro/paginação em memória na rota de jogadores.	Custo de CPU e memória cresce com o volume total; risco de timeout em catálogo grande.
Alta	backend/products/serializers.py:7-31	Serializer aninhado do jogador na listagem de produtos.	Payload maior e mais CPU em serialização; latência aumentada em páginas com muitos itens.

Média	frontend/src/features/players/hooks.ts:35-44	Polling redundante de leilões no frontend.	Aumento de trafego e requisições sem necessidade em múltiplas telas.
Ponto forte	backend/core/settings/api.py:3-8	Paginação padrão do DRF habilitada.	Limita a carga por página e reduz pressão em banco e frontend.

Recomendações priorizadas

1. P1 — Manter paginação do backend e evitar carregar a coleção completa; seguir com filtros no servidor sempre que possível.

2. P1 — Continuar a serialização leve para listagens de produto e leilão; manter os serializers completos apenas em detalhes e endpoints específicos.

3. P2 — Reduzir polling indiscriminado de leilões e usar atualização por evento ou frequência menor em telas ativas.

4. P3 — Medir a lista em massa com um dataset real e validar o tempo de resposta nas rotas de catalogo e página de jogadores.

ISSUES PARA O GITHUB

--- ISSUE 1 ---

[Performance] Listagem de jogadores faz filtro e paginação em memória no frontend

Labels sugeridas: performance + crítica Descrição do problema: A rota de jogadores carregava o catálogo completo na memória e aplicava filtros e ordenação localmente antes da renderização. Isso gera custo de CPU e memória que cresce desproporcionalmente com o número de itens. Evidência: frontend/src/routes/players.index.tsx:29-58; frontend/src/components/PlayerFilters.tsx:92-164. Impacto estimado: com 5.000 itens, cada render percorre a lista completa e a UI fica mais lenta, com risco de timeout. Sugestão de correção: usar paginação do backend e aplicar filtros/ordenação no servidor sempre que possível. Critérios de aceite: [ ] a página usa backend pagination; [ ] a lista trabalha com page_size; [ ] o tempo de render continua responsivo com 1.000+ itens.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Performance] Serialização aninhada do jogador em cada item de produto e leilão

Labels sugeridas: performance + alta Descrição do problema: os serializers de produto e leilão expandiam o objeto do jogador completo, incluindo o campo photo_data_uri e dados redundantes. Isso inflava o payload e aumentava o custo de serialização. Evidência: backend/products/serializers.py:7-31; backend/fifa_data/serializers.py:7-35; backend/auction/serializers.py:5-16. Impacto estimado: páginas com vários itens geram payloads maiores, maior latência e mais gasto de CPU no backend. Sugestão de correção: manter um serializer leve para listagem e reduzir o conjunto de campos. Critérios de aceite: [ ] /products/ retorna só campos essenciais; [ ] detalhe continua completo; [ ] payload da página cai significativamente.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Performance] Polling redundante de leilões no frontend

Labels sugeridas: performance + média Descrição do problema: o hook de leilões usa refetch periódico em intervalo fixo, mesmo quando a tela principal não precisa de atualização em tempo real. Evidência: frontend/src/features/players/hooks.ts:35-44. Impacto estimado: múltiplas telas abertas podem multiplicar requisições redundantes e consumir tráfego desnecessário. Sugestão de correção: reduzir ou condicionar o polling e, quando necessário, usar eventos/websocket. Critérios de aceite: [ ] polling reduzido ou condicionado; [ ] requisições por minuto caem; [ ] a tela de jogadores continua funcional sem polling global.

--- FIM ISSUE 3 ---